



COMP2000 Week 2: Git

The goal of this week is to understand distributed version control with Git. You will create repositories, make commits, inspect history, and synchronise changes between team members. By treating commits as *chat messages* you will simulate a messaging system using only Git commands.

Topics

Repository management Creating a new repository with `git init` and copying an existing one with `git clone`.

Tracking changes The three-tree model (working directory, staging area, repository) and the `add/commit` workflow.

History inspection Reading a project's history with `git log` and identifying contributions with `git blame`.

Synchronisation Transferring commits between repositories with `fetch`, `pull`, and `push`.

Preparation

Attend the lecture in week 1.

Read the following Atlassian Git tutorial pages. They form a complete track from first principles through to collaboration:

1. What is version control¹
2. What is Git²
3. Setting up a repository³
4. Saving changes⁴
5. Inspecting a repository⁵
6. Syncing⁶

Make sure you understand each command's purpose before class. The EPIC activity will ask you to use all of them.

¹<https://www.atlassian.com/git/tutorials/what-is-version-control>

²<https://www.atlassian.com/git/tutorials/what-is-git>

³<https://www.atlassian.com/git/tutorials/setting-up-a-repository>

⁴<https://www.atlassian.com/git/tutorials/saving-changes>

⁵<https://www.atlassian.com/git/tutorials/inspecting-a-repository>

⁶<https://www.atlassian.com/git/tutorials/syncing>



EPIC activity

Your task this week is to *chat with your team members using Git as a chat server*. This will take some serious imagination! You will need a strong understanding of the following commands:

- `git init / git clone`
- `git add / git commit`
- `git push / git pull / git fetch`
- `git log`

How it works. Treat each commit as a chat message. The commit message is the text of your message; the files you include are your attachments. You send a message by committing and pushing. You check for new messages by fetching and viewing the log.

1. One team member creates a shared repository on GitHub/Bitbucket/GitLab and adds the others as collaborators.
2. Each member clones the repository to their own computer.
3. Members take turns composing and pushing commit messages. See if you can do all communication with git instead of talking in person!
4. Use `git log` to read the conversation history.
5. Experiment with features: multi-line messages, file attachments (images, text), replies as new commits, timestamps.
6. Try to achieve *completely private chat with no central server* using only Git's built-in mechanisms.

Your team will be scored on: the number of messages, the complexity of messages, and the number of chat features you have emulated. The class teacher will track scores on a whiteboard during class.

teacher's note

Circulate around class and keep them on track. Put up a table on the whiteboard during the EPIC activity and periodically update the scores based on your observations. For specific reporting, have the one person from each group move to the front of class and project. Other team members stay at their desks and participate in the demo. The demo period is a chance for the team to finalise its scores in the competition. Please grab a photo of the final scoreboard and post it to the forums.

Reporting

When the teacher calls time, one person from each team moves to the front of class and projects the team's chat log. It will be a compelling demo if you can chat to your teammates who are still sitting at their desks. Use the demo period to finalise your team's scores in the competition. Scores are tracked on the whiteboard based on the number of messages, their complexity, and the number of chat features emulated.

Your log book should record the final scoreboard and the most creative approach you saw from another team.



Self Study Exercises

1. List three main benefits of version control systems. For each benefit, explain how it applies to the team chat scenario from this week's exercise.
2. Explain what “distributed” means in the context of Git. In the chat exercise, is it possible to communicate without a central server on GitHub? How?
3. What is the difference between `git init` and `git clone`? In the chat exercise, which command would you use to start a new chat room, and which would you use to join an existing one?
4. Name Git's three trees (also called areas). Where is a message after you have typed it into a file? After `git add`? After `git commit`?
5. Why does Git have a staging area? How could you use it to compose a multi-part message (e.g. text plus an image) before sending it as a single commit?
6. Write a `git log` command that shows each commit on a single line with a text-based graph of branches. How would you use this command to read the conversation in your chat repository?
7. What is the difference between `git fetch` and `git pull`? If you want to check for new messages without automatically merging them into your local history, which command should you use?
8. Name the two most common protocols used for Git remote URLs. If your team wanted a completely private chat with no central server, what could you use in place of a URL like `https://github.com/...`?